

# CropWise

A crop recommendation system that suggests the 3 best crops for a field from its soil and climate, explained step by step.

**2,200**

rows in the dataset

**22**

crops (classes), 100  
rows each

**7**

input features

**99.5%**

Random Forest test  
accuracy

1. Problem and solution
2. The dataset
3. Preparing the data
4. The models we compared
5. How the Random Forest works
6. How the top 3 predictions are made
7. Results and evaluation
8. Confidence demo: the 3 samples
9. The web application
10. Limitations and future work
11. Viva questions and answers

Live site: [crop-prediction-amber.vercel.app](https://crop-prediction-amber.vercel.app) · Code: [github.com/mihirr-spec/Crop-Prediction](https://github.com/mihirr-spec/Crop-Prediction)

# 1. Problem and solution

Many farmers choose a crop by habit or by what neighbours grow, not by matching it to their own soil and climate. A poor match means lower yield and wasted fertilizer and water.

**Our solution:** the farmer enters 7 values, and a machine-learning model trained on 2,200 examples recommends the 3 crops most likely to suit those conditions, each with a confidence score.

**Type of problem:** supervised learning, **multi-class classification**. The inputs are 7 numbers, and the output is one of 22 crop labels.

## 2. The dataset

**Source:** Kaggle “Crop Recommendation Dataset” (Atharva Ingle), built from Indian rainfall, climate and soil-fertilizer data. File: `data/Crop_recommendation.csv`.

**Size:** 2,200 rows × 8 columns (7 features + 1 label). **No missing values, no duplicate rows.**

| Column             | Meaning                   | Unit   | Min  | Max   | Mean  |
|--------------------|---------------------------|--------|------|-------|-------|
| <b>N</b>           | Nitrogen in the soil      | kg/ha* | 0.0  | 140.0 | 50.6  |
| <b>P</b>           | Phosphorus in the soil    | kg/ha* | 5.0  | 145.0 | 53.4  |
| <b>K</b>           | Potassium in the soil     | kg/ha* | 5.0  | 205.0 | 48.1  |
| <b>temperature</b> | Air temperature           | °C     | 8.8  | 43.7  | 25.6  |
| <b>humidity</b>    | Relative humidity         | %      | 14.3 | 100.0 | 71.5  |
| <b>ph</b>          | Soil acidity / alkalinity | 0–14   | 3.5  | 9.9   | 6.5   |
| <b>rainfall</b>    | Rainfall                  | mm     | 20.2 | 298.6 | 103.5 |

\* The dataset describes N, P and K as the nutrient content of the soil without stating a unit; the website labels them kg/ha, as on an Indian Soil Health Card. The dataset also does not say which period rainfall covers.

### The 22 classes (crops)

Each crop has exactly **100 rows**, so the dataset is **perfectly balanced**, and no crop dominates the training.

| Group                 | Crops                                                                                     |
|-----------------------|-------------------------------------------------------------------------------------------|
| <b>Cereals (2)</b>    | Rice, Maize                                                                               |
| <b>Pulses (7)</b>     | Chickpea, Kidney Beans, Pigeon Peas, Moth Beans, Mung Bean, Black Gram, Lentil            |
| <b>Fruits (10)</b>    | Mango, Banana, Papaya, Coconut, Grapes, Apple, Orange, Pomegranate, Watermelon, Muskmelon |
| <b>Cash crops (3)</b> | Cotton, Jute, Coffee                                                                      |

## Average conditions for each crop

The mean of the 100 rows for each crop. Crops with very different averages, such as rice and chickpea, are easy to tell apart. Crops with similar averages, such as rice and jute, are where the model hesitates.

| Crop         | N   | P   | K   | Temp °C | Humidity % | pH  | Rain mm |
|--------------|-----|-----|-----|---------|------------|-----|---------|
| Rice         | 80  | 48  | 40  | 23.7    | 82         | 6.4 | 236     |
| Maize        | 78  | 48  | 20  | 22.4    | 65         | 6.2 | 85      |
| Chickpea     | 40  | 68  | 80  | 18.9    | 17         | 7.3 | 80      |
| Kidney Beans | 21  | 68  | 20  | 20.1    | 22         | 5.7 | 106     |
| Pigeon Peas  | 21  | 68  | 20  | 27.7    | 48         | 5.8 | 150     |
| Moth Beans   | 21  | 48  | 20  | 28.2    | 53         | 6.8 | 51      |
| Mung Bean    | 21  | 47  | 20  | 28.5    | 86         | 6.7 | 48      |
| Black Gram   | 40  | 68  | 19  | 30.0    | 65         | 7.1 | 68      |
| Lentil       | 19  | 68  | 19  | 24.5    | 65         | 6.9 | 46      |
| Mango        | 20  | 27  | 30  | 31.2    | 50         | 5.8 | 95      |
| Banana       | 100 | 82  | 50  | 27.4    | 80         | 6.0 | 105     |
| Papaya       | 50  | 59  | 50  | 33.7    | 92         | 6.7 | 143     |
| Coconut      | 22  | 17  | 31  | 27.4    | 95         | 6.0 | 176     |
| Grapes       | 23  | 132 | 200 | 23.8    | 82         | 6.0 | 70      |
| Apple        | 21  | 134 | 200 | 22.6    | 92         | 5.9 | 113     |
| Orange       | 20  | 17  | 10  | 22.8    | 92         | 7.0 | 110     |
| Pomegranate  | 19  | 19  | 40  | 21.8    | 90         | 6.4 | 108     |
| Watermelon   | 99  | 17  | 50  | 25.6    | 85         | 6.5 | 51      |
| Muskmelon    | 100 | 18  | 50  | 28.7    | 92         | 6.4 | 25      |
| Cotton       | 118 | 46  | 20  | 24.0    | 80         | 6.9 | 80      |
| Jute         | 78  | 47  | 40  | 25.0    | 80         | 6.7 | 175     |
| Coffee       | 101 | 29  | 30  | 25.5    | 59         | 6.8 | 158     |

### 3. Preparing the data

1. **Separate inputs and label:** `x` = the 7 feature columns, `y` = the crop name.
2. **Train/test split:** 80% training (1760 rows), 20% testing (440 rows). `stratify=y` keeps exactly 80 training and 20 test rows per crop, and `random_state=42` makes the split repeatable.
3. **Scaling (Logistic Regression only):** `StandardScaler` rescales each feature to mean 0 and standard deviation 1. It is fitted on the training data only, so no information from the test set leaks into training. The Random Forest uses the raw values, because tree splits do not depend on scale.

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, stratify=y, random_state=42)
```

### 4. The models we compared

| Model                         | Train accuracy | Test accuracy | Micro AUC |
|-------------------------------|----------------|---------------|-----------|
| Logistic Regression (scaled)  | 97.4%          | 97.3%         | 0.9999    |
| <b>Random Forest (chosen)</b> | 100.0%         | <b>99.5%</b>  | 1.0000    |

**Tuning the Random Forest:** `GridSearchCV` tried 6 combinations, `n_estimators`  $\in \{100, 200\} \times \text{max\_depth} \in \{\text{None}, 10, 20\}$ , each with 5-fold cross-validation. Best: **`n_estimators = 200`, `max_depth = None`**, with a cross-validation accuracy of **99.4%**.

**Why Random Forest?** It learns non-linear boundaries and feature interactions, needs no scaling, copes well with features of very different ranges, and gives a probability for every crop, which we need for the top 3. Logistic Regression only draws straight-line boundaries.

## 5. How the Random Forest works

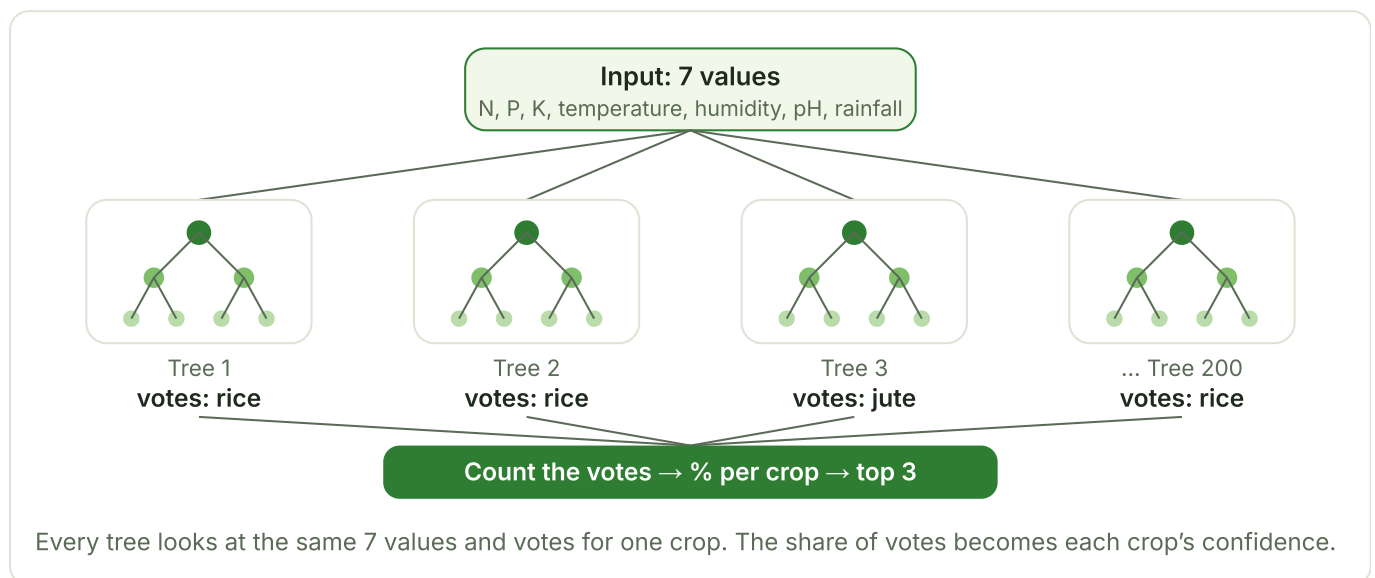
A Random Forest is a group ("ensemble") of decision trees that vote.

### One decision tree

A tree asks a series of yes/no questions such as *"Is rainfall  $\leq 120$  mm?"* or *"Is humidity  $\leq 30\%$ ?"*. At each step it chooses the question that best separates the crops, measured by **Gini impurity**, which is how mixed the crops are in a group (0 = all one crop). It keeps splitting until each leaf holds only one crop. Our trees are about 15 questions deep with about 75 leaves each.

### Why many trees, and why "random"

- **Bootstrap sampling:** each tree is trained on a random sample of the 1,760 training rows, drawn with replacement, so every tree sees slightly different data.
- **Random features:** at each split a tree may only consider a random subset of features, `max_features='sqrt'`, which means 2 of the 7. This stops all trees from asking the same questions.
- **Voting:** one tree can be wrong, but the majority of 200 different trees is rarely wrong. This is called **bagging**, and it reduces overfitting.



### Model settings used

| Setting                   | Value | Meaning                                 |
|---------------------------|-------|-----------------------------------------|
| <code>n_estimators</code> | 200   | number of trees                         |
| <code>max_depth</code>    | None  | trees grow until every leaf is pure     |
| <code>criterion</code>    | gini  | how split quality is measured           |
| <code>max_features</code> | sqrt  | 2 of the 7 features tried at each split |
| <code>bootstrap</code>    | True  | each tree gets a random sample of rows  |
| <code>random_state</code> | 42    | makes training repeatable               |

## 6. How the top 3 predictions are made

1. The 7 values are put in the same column order as in training: N, P, K, temperature, humidity, ph, rainfall .
2. `model.predict_proba(X)` returns 22 numbers, one probability per crop, which add up to 1. scikit-learn averages the class probabilities given by all 200 trees. Our trees grow until their leaves are pure, so each tree gives 100% to a single crop, and the **probability equals the share of trees that voted for that crop**.
3. We sort the 22 probabilities from high to low and keep the first 3.
4. Any value outside the training range is reported as a warning.

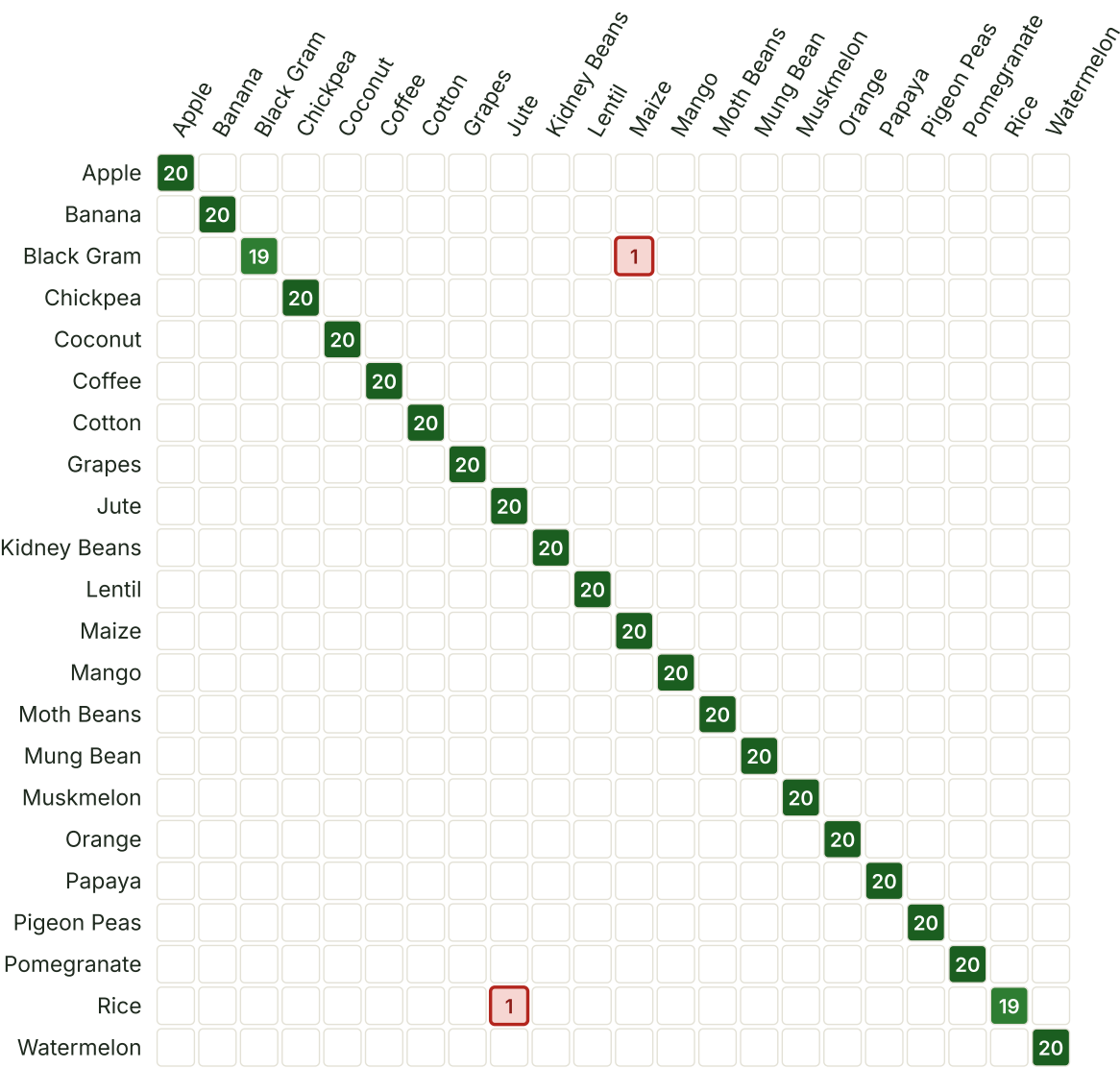
```
# cropwise/predict.py (simplified)
proba = model.predict_proba(X)[0]          # 22 probabilities
top_idx = proba.argsort()[::-1][:3]         # indices of the 3 highest
top = [{"crop": model.classes_[i], "confidence": proba[i]} for i in top_idx]
```

**Proof that probability = votes:** for Sample 3 we counted the trees one by one: **105 voted Jute** and **95 voted Rice**.  $105/200 = 52.5\%$  and  $95/200 = 47.5\%$ , exactly the confidence shown on the website.

**What the score means:** "85% match" means 170 of the 200 trees chose that crop. It is the model's *confidence*, not a guarantee of a good harvest.

# 7. Results and evaluation

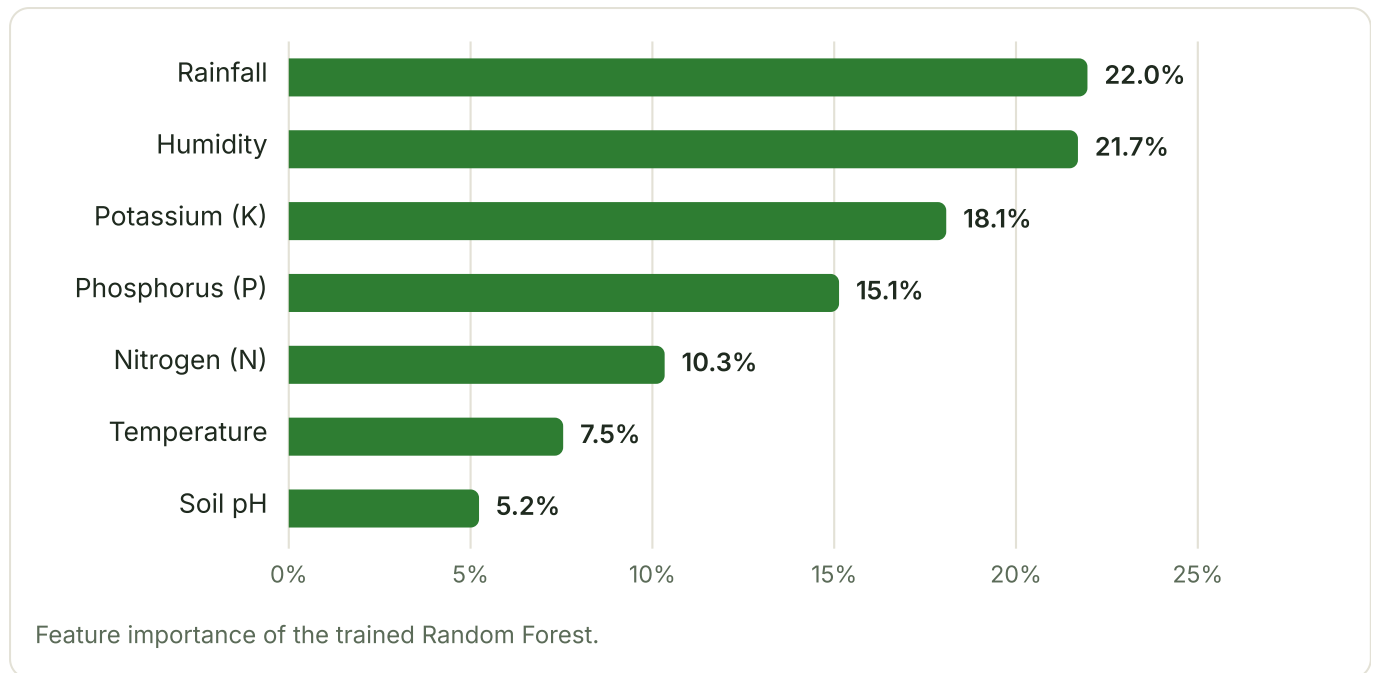
On the 440 test rows the model has never seen, the Random Forest is right **438 times out of 440 (99.5%)**. Macro-average F1 score: **0.995**. The lowest per-crop F1 scores are Black Gram 0.974, Rice 0.974. Every other crop scores 1.000.



Confusion matrix on the test set (rows = true crop, columns = predicted crop). The diagonal holds the correct predictions (20 each); the red cells are the 2 mistakes: Black Gram predicted as Maize; Rice predicted as Jute.

## Which features matter most?

Feature importance measures how much each feature reduces Gini impurity across all trees in the forest. The 7 values add up to 100%.



**Rainfall** and **humidity** separate the crops best, for example wet rice and jute against dry chickpea and moth beans. **pH** matters least because most crops grow in a similar pH range of about 5.5–7.5.

## ROC / AUC

For each crop we treat “this crop or not” as a yes/no problem and plot the true-positive rate against the false-positive rate. Micro-averaged AUC is **1.0000** for Random Forest and **0.9999** for Logistic Regression (1.0 is perfect).

## Is it overfitting?

Train accuracy is 100.0% because fully grown trees memorise their training rows, which is normal for a Random Forest. On unseen data, test accuracy is 99.5% and 5-fold cross-validation is 99.4%, almost as high. So the model generalises well and is **not** overfitting in a harmful way.

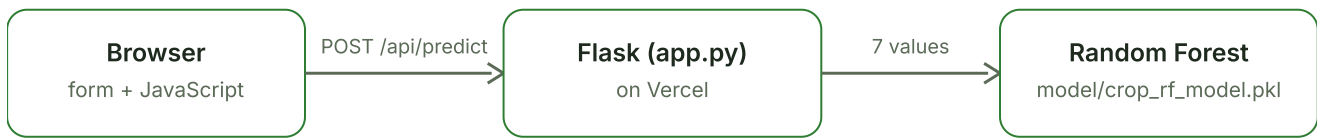
## 8. Confidence demo: the 3 samples

The website has 3 sample buttons. Sample 1 is a real row from the dataset. Samples 2 and 3 change only rainfall and temperature, to show how the model's confidence falls as conditions move from "typical rice" towards "between rice and jute".

| Sample                                                                                                                                                                 | Values                                                   | Tree votes (out of 200)                                                  |
|------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------|--------------------------------------------------------------------------|
| <b>Sample 1</b><br>Exact row from the dataset (a rice field)                                                                                                           | N 91, P 50, K 40<br>20.8 °C, 84.1 %, pH 6.5,<br>230.2 mm | <div>Rice 199</div> <div>Rice 99.5%, Jute 0.5%</div>                     |
| The model has learned from many fields like this one, so 199 of the 200 trees vote rice.                                                                               |                                                          |                                                                          |
| <b>Sample 2</b><br>Same field, rainfall 230.2 → 195 mm                                                                                                                 | N 91, P 50, K 40<br>20.8 °C, 84.1 %, pH 6.5,<br>195.0 mm | <div>Rice 171</div> <div>Rice 85.5%, Jute 14.5%</div>                    |
| 195 mm is drier than most rice rows (average 236 mm) and closer to jute (average 175 mm). Jute grows in the same warm, humid conditions, so some trees switch to jute. |                                                          |                                                                          |
| <b>Sample 3</b><br>Rainfall 190 mm and temperature 20.8 → 23 °C                                                                                                        | N 91, P 50, K 40<br>23.0 °C, 84.1 %, pH 6.5,<br>190.0 mm | <div>Jute 105</div> <div>Rice 95</div> <div>Jute 52.5%, Rice 47.5%</div> |
| Now the values sit right between rice and jute. The trees split almost evenly. With only two close crops competing, the top score cannot drop much below 50%.          |                                                          |                                                                          |

**Key idea:** high confidence means the field looks like many training examples of one crop. Low confidence means the field sits between crops that grow in similar conditions, here rice and jute, which are also the pair the model confused once in the test set.

## 9. The web application



The browser sends the 7 values to Flask, which asks the model and returns the top 3 crops as JSON.

### Pages

- **Home:** hero, gallery of the 22 crops by group, and the Predict Crop form.
- **/recommend:** the Predict Crop form on its own.
- **/credits:** photo authors and licences.

### API

```
POST /api/predict
{"N": 91, "P": 50, "K": 40, "temperature": 20.8,
 "humidity": 84.1, "ph": 6.5, "rainfall": 230.2}
→ {"top": [{"crop": "rice", "confidence": 0.995}, ...],
   "warnings": []}
```

### Technology

- Python, pandas, scikit-learn, joblib
- Flask (backend), HTML/CSS/JavaScript (frontend)
- Vercel hosting, GitHub auto-deploy

### Languages

English, Hindi, Odia, Bengali, Tamil, Telugu, Kannada and Malayalam. All text is in `cropwise/i18n.py`, the language is chosen with `?lang=` and remembered in a cookie, and each script uses a matching Noto Sans font.

### Mobile

Phone menu, large touch targets, number keypad for inputs, checked at 360 px width in every language.

### Project structure

|                                      |                                              |
|--------------------------------------|----------------------------------------------|
| <code>app.py</code>                  | Flask app: pages + <code>/api/predict</code> |
| <code>cropwise/predict.py</code>     | loads the model, returns the top 3           |
| <code>cropwise/crops.py</code>       | crop groups, averages, sample values         |
| <code>cropwise/i18n.py</code>        | website text in 8 languages                  |
| <code>model/crop_rf_model.pkl</code> | trained Random Forest                        |
| <code>data/</code>                   | <code>Crop_recommendation.csv</code>         |
| <code>notebooks/</code>              | training and evaluation notebook             |
| <code>templates/, static/</code>     | HTML, CSS, JavaScript, crop photos           |

## 10. Limitations and future work

### Limitations

- Only 22 crops and 2,200 rows; the data is cleaner and more separable than real farm data, which is why accuracy is so high.
- The dataset does not state the units of N, P and K or the period that rainfall covers.
- N, P, K and pH have to come from a soil test (for example a Soil Health Card).
- Season, soil type, irrigation, pests and market prices are not considered.
- The score is the model's confidence, not a yield guarantee; values outside the training range give less reliable answers.

### Future work

- Train on more real Indian field data, with more crops and regions.
- Add season, soil type and irrigation as features.
- Suggest fertilizer to fix N, P or K shortfalls for the chosen crop.
- Calibrate the probabilities, and have native speakers review the translations.

# 11. Viva questions and answers

## 1. What is the aim of the project?

To recommend the crops most suited to a field, using 7 measurable values: soil N, P, K and pH, plus temperature, humidity and rainfall. The website shows the top 3 crops with a confidence score, in 8 Indian languages.

## 2. Which dataset did you use?

The Kaggle "Crop Recommendation Dataset" (by Atharva Ingle). It was built from Indian rainfall, climate and soil-fertilizer data. It has 2,200 rows, 7 input columns and 1 label column.

## 3. How many classes are there?

22 crops, with exactly 100 rows each, so the classes are perfectly balanced. This is a multi-class classification problem.

## 4. Was any cleaning needed?

No. There are no missing values and no duplicate rows (checked in the notebook with `df.info()` and `df.duplicated()`). All 7 inputs are numbers, so no encoding was needed for them. The label is text, which scikit-learn handles directly.

## 5. How did you split the data?

80% for training (1760 rows) and 20% for testing (440 rows), with `stratify=y` so every crop has 80 train and 20 test rows, and `random_state=42` so the split is repeatable.

## 6. Which ML model is behind the website?

A Random Forest classifier from scikit-learn with 200 decision trees, `max_depth=None` (trees grow until their leaves are pure), Gini impurity for splits, bootstrap sampling, and `max_features='sqrt'` (2 of the 7 features are tried at each split).

## 7. Why Random Forest and not Logistic Regression?

We trained both. Logistic Regression draws straight-line (linear) boundaries and got 97.3% test accuracy. Random Forest can learn curved, non-linear boundaries and interactions between features, and got 99.5%. It also needs no feature scaling and gives a probability for every crop, which we use for the top 3.

## 8. How does a Random Forest work?

Each tree is trained on a random bootstrap sample of the training rows (drawn with replacement), and at every split it may only look at a random subset of features. That makes the trees different from each other. To predict, every tree votes for one crop and the forest combines the votes. Many varied trees make fewer mistakes than one tree.

## 9. How does one decision tree decide?

It asks yes/no questions about the features, such as "rainfall  $\leq$  120 mm?". At each step it picks the question that best separates the crops, measured by Gini impurity (how mixed the crops are in each group). It keeps splitting until each leaf holds only one crop.

## 10. How do you get the top 3 predictions?

We call `model.predict_proba()`, which gives a probability for each of the 22 crops. scikit-learn averages the class probabilities from all 200 trees. Because our trees grow until leaves are pure, each tree gives 100% to one crop, so the probability equals the share of trees voting for it. We sort the 22 probabilities and keep the highest 3 (`argsort` in `predict.py`).

### **11. What does “85% match” mean?**

170 of the 200 trees voted for that crop. It is the model’s confidence, not a guarantee that the crop will grow well.

### **12. Is 100% train accuracy overfitting?**

Train accuracy is 100% because fully grown trees memorise their training rows, which is normal for Random Forests. What matters is unseen data: 99.5% on the test set and 99.4% in 5-fold cross-validation. Both are almost as high, so the model generalises well.

### **13. What hyperparameters did you tune?**

n\_estimators (100 or 200 trees) and max\_depth (None, 10 or 20), with GridSearchCV and 5-fold cross-validation on the training set. The best was n\_estimators=200, max\_depth=None.

### **14. What is cross-validation?**

The training set is split into 5 parts. The model is trained on 4 and tested on the 5th, five times, each time with a different part held out. The average score is a fairer estimate than one split.

### **15. Why did you scale the features only for Logistic Regression?**

Logistic Regression is sensitive to feature scale (rainfall is in the hundreds, pH below 10). StandardScaler gives every feature mean 0 and standard deviation 1. A tree only compares one feature against a threshold, so scale does not change it, and the Random Forest uses raw values. The scaler is fitted on training data only, so no test information leaks in.

### **16. Which features matter most?**

From the forest’s feature importances: rainfall (22.0%) and humidity (21.7%) matter most, then potassium, phosphorus and nitrogen. pH matters least (5.2%).

### **17. Where does the model make mistakes?**

Only 2 of 440 test rows are wrong: one Black Gram predicted as Maize; one Rice predicted as Jute. These crops grow in similar conditions, which is also why Sample 3 is split between rice and jute.

### **18. What is the ROC curve / AUC?**

For each crop we treat “this crop or not” as a yes/no problem and plot true-positive rate against false-positive rate at every threshold. Micro-averaged AUC is 1.0000 for Random Forest and 0.9999 for Logistic Regression. 1.0 is perfect.

### **19. Why is the accuracy so high?**

The dataset is clean, balanced and its crops have quite distinct average conditions, so it is an easy problem compared with messy real farm data. Real fields would need more data, and the score would likely be lower.

### **20. What happens with values outside the dataset’s range?**

The model still predicts, but the website shows a warning naming the value that is outside the range seen in training (for example rainfall above 298.6 mm), because the model has no examples there.

### **21. How does the website work?**

The browser sends the 7 values as JSON to POST /api/predict. Flask passes them to recommend\_top\_k() in cropwise/predict.py, which loads the saved model once and returns the top 3 crops and any warnings. The page then shows them with photos.

## **22. How is the site multilingual?**

All text lives in `cropwise/i18n.py` in 8 languages: English, Hindi, Odia, Bengali, Tamil, Telugu, Kannada and Malayalam. The chosen language comes from `?lang=` and is remembered in a cookie. Each script uses a matching Noto Sans font.

## **23. How is it deployed?**

On Vercel as a Python (Flask) serverless app, connected to the GitHub repository. Every push to main redeploys automatically.

## **24. What are the limitations?**

Only 22 crops and 2,200 rows. The dataset does not say what period rainfall covers. N, P and K must come from a soil test, and the model ignores factors like irrigation, market price and pests. The score is the model's confidence, not a yield guarantee.

## **25. How could it be improved?**

More real Indian field data and more crops; adding season, soil type and irrigation as features; fertilizer suggestions; calibrated probabilities; and native-speaker review of the translations.